이산수학
Report 2

인공지능학부 20233144 변수양

Github: https://github.com/suyamg/discrete_mathematics

Report2.py

목차

1. 문제 정의 및 구성 조건
2. 프로그램 전체 구조
3. 각 구성 조건 만족 여부(실행 단계)
4. 추가 기능
5. 테스트

문제 정의

집합 A = {1,2,3,4,5}에 관한 관계행렬을 입력 받아서,
이 관계가 동치 관계임을 판별하고, 동치 관계이면 동치류를 출력한다.

구성 조건

1. 관계 행렬 입력 기능
2. 동치 관계 판별 기능
3. 동치 관계일 경우 동치류 출력 기능
4. 폐포 구현 기능

구성 조건 (1)

관계 행렬 입력 기능 조건

1) 사용자로부터 5×5 크기의 정방행렬을 행 단위로 입력 받을 수 있어야 함
2) 입력 데이터는 리스트(2차원 배열)로 저장

구성 조건 (2)

동치 관계 판별 기능 조건

1) 반사, 대칭, 추이 관계를 각각 판별하는 함수를 구현 입력
2) 관계의 성질 판별 결과에 따라 동치 관계 여부에 대한 메시지를 출력

구성 조건 (3)

동치 관계일 경우 결과에 따라 동치 관계 여부에 대한 메시지 출력 조건

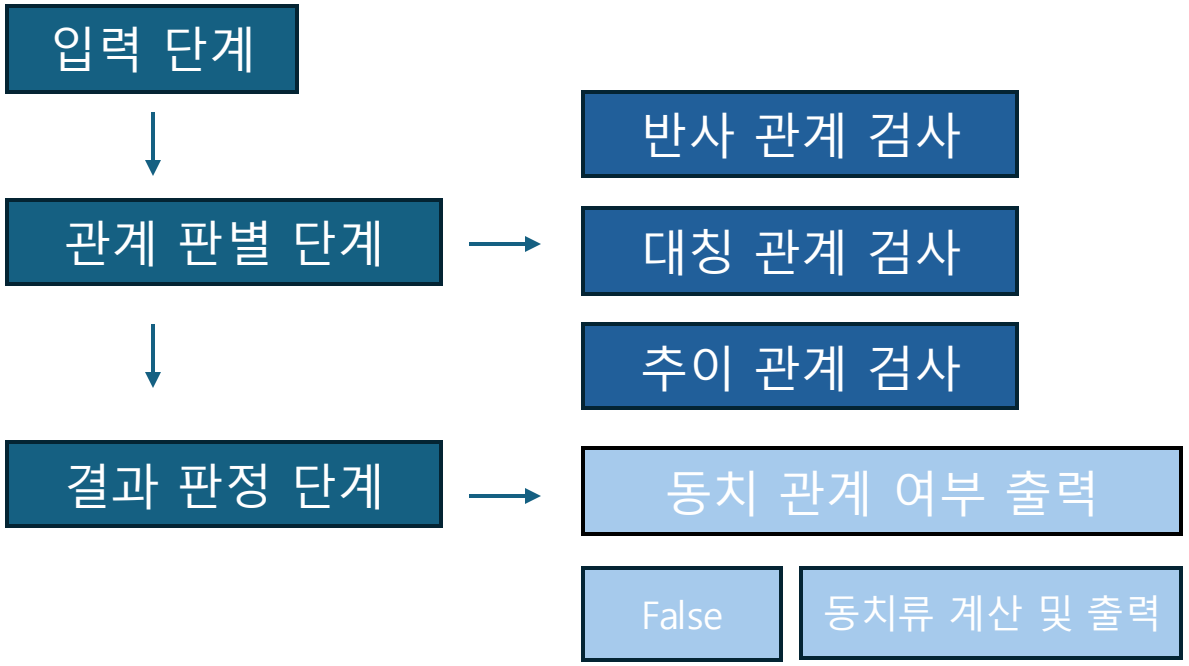
1) 집합의 원소에 대해 동치류를 판별하는 함수를 구현
2) 집합의 각 원소에 대한 동치류를 각각 출력

구성 조건 (4)

폐포 구현 기능 조건

1) 입력 받은 관계가 반사, 대칭, 추이 관계가 아닐 경우
   각각의 폐포로 만드는 함수를 구현
2) 각각의 관계에 대한 폐포 변환 전, 변환 후를 출력
3) 각각의 폐포로 변환한 후 동치 관계를 다시 판별하고 동치류 출력하기

프로그램 전체 구조



[입력]
└── 관계 행렬 (5×5)

[관계 판별 모듈]
├── 반사성 검사 함수 (is_reflexive)
├── 대칭성 검사 함수 (is_symmetric)
├── 추이성 검사 함수 (is_transitive)

[폐포 변환 모듈]
├── reflexive_closure()
├── symmetric_closure()
├── transitive_closure()

[동치 관계 판별]
└── check_relation()

[동치류 계산]
└── equivalence_classes()

각 구성 조건 만족 여부

1. 관계 행렬 입력 기능 ✅
2. 동치 관계 판별 기능 ✅
3. 동치 관계일 경우 동치류 출력 기능 ✅
4. 폐포 구현 기능 ✅

구성 조건 (1) 관계 행렬 입력 기능 ✅

```python
def get_matrix():
    R = []
    for i in range(5):
        row = []
        for j in range(5):
            val = matrix_entries[i][j].get().strip()
            if val == "":
                messagebox.showwarning("입력 오류", "모든 칸에 값을 입력하세요.")
                return None
            row.append(int(val))
        R.append(row)
    return R
```

사용자가 집합 A = {1,2,3,4,5}에 대한 5x5 관계 행렬을 직접 입력하는 기능
: 각 원소 간의 관계를 0과 1로 입력 받아 프로그램의 판별 과정에 사용

구성 조건 (2) 동치 관계 판별 기능 ✅

```python
def check_relation():
    R = get_matrix()
    if R is None: return
    clear_results()
    reflexive = is_reflexive(R)
    symmetric = is_symmetric(R)
    transitive = is_transitive(R)

    text = f"반사적: {reflexive}\n대칭적: {symmetric}\n추이적: {transitive}"
    label = tk.Label(result_frame, text=text, font=("Apple SD Gothic Neo", 12,
    "bold"), bg="#eef6f9", fg="#333")
    label.pack(pady=5)

    if reflexive and symmetric and transitive:
        tk.Label(result_frame, text="✅ 이 관계는 동치 관계입니다!", fg="green",
                 font=("Apple SD Gothic Neo", 10, "bold"), bg="#eef6f9").pack()
        classes = equivalence_classes(R)
        for i, eq in enumerate(classes):
            tk.Label(result_frame, text=f"{A[i]}의 동치류: {eq}",
                     font=("Apple SD Gothic Neo", 11), bg="#eef6f9").pack()
    else:
        tk.Label(result_frame, text="❌ 동치 관계가 아닙니다.", fg="red",
                 font=("Apple SD Gothic Neo", 10, "bold"), bg="#eef6f9").pack()
```

입력된 관계 행렬에 대해 반사성, 대칭성, 추이성을 각각 검사

- 세 조건이 모두 만족될 경우 동치 관계로 판정
- 결과를 화면에 메시지로 출력

구성 조건 (3) 동치 관계일 때 출력 기능 ✅

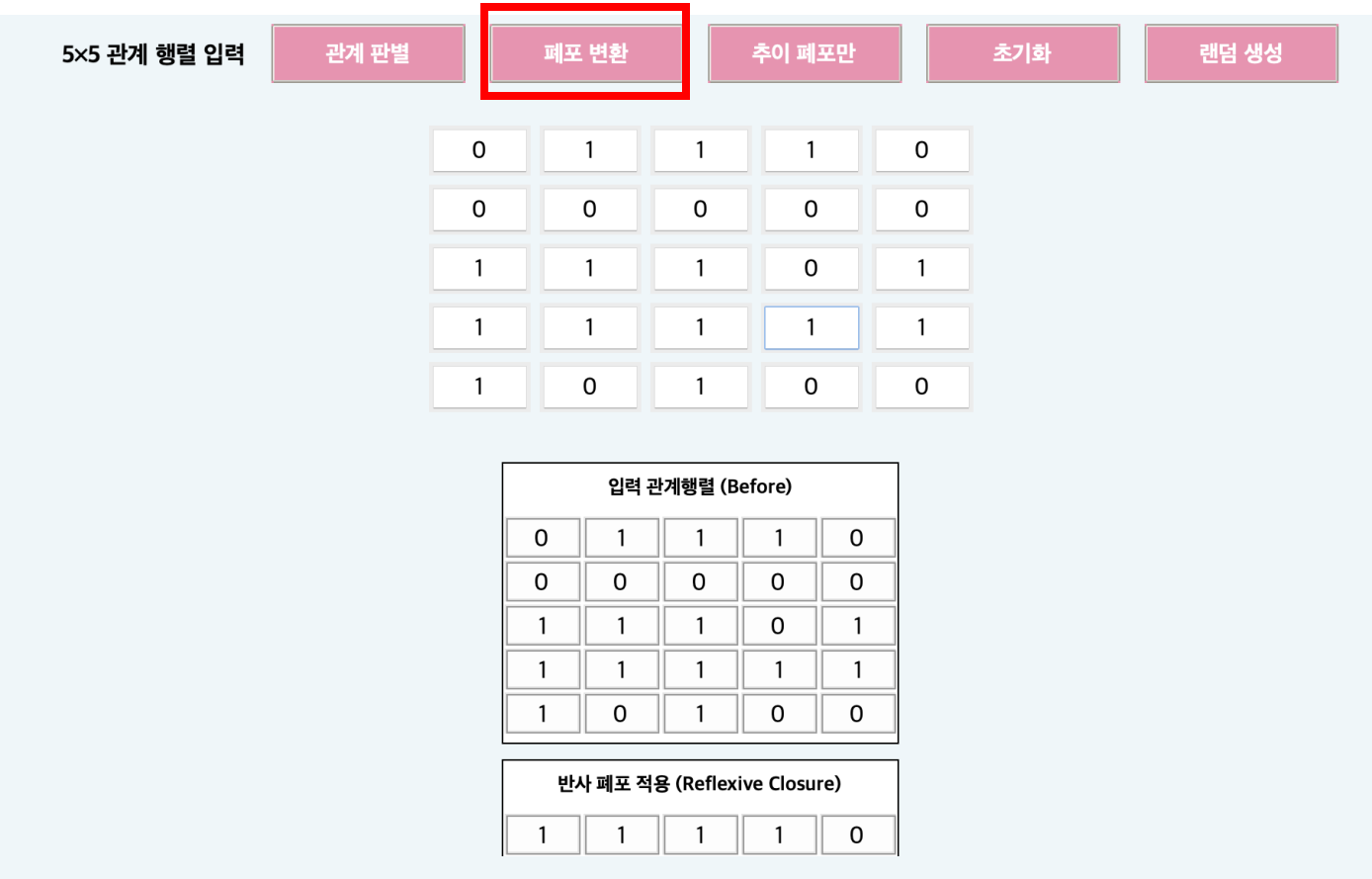
```python
def check_relation():
    R = get_matrix()
    if R is None: return
    clear_results()
    reflexive = is_reflexive(R)
    symmetric = is_symmetric(R)
    transitive = is_transitive(R)

    text = f"반사적: {reflexive}\n대칭적: {symmetric}\n추이적: {transitive}"
    label = tk.Label(result_frame, text=text, font=("Apple SD Gothic Neo", 12,
    "bold"), bg="#eef6f9", fg="#333")
    label.pack(pady=5)

    if reflexive and symmetric and transitive:
        tk.Label(result_frame, text="✅ 이 관계는 동치 관계입니다!", fg="green",
                 font=("Apple SD Gothic Neo", 10, "bold"), bg="#eef6f9").pack()
        classes = equivalence_classes(R)
        for i, eq in enumerate(classes):
            tk.Label(result_frame, text=f"{A[i]}의 동치류: {eq}",
                     font=("Apple SD Gothic Neo", 11), bg="#eef6f9").pack()
    else:
        tk.Label(result_frame, text="❌ 동치 관계가 아닙니다.", fg="red",
                 font=("Apple SD Gothic Neo", 10, "bold"), bg="#eef6f9").pack()
```

```python
def equivalence_classes(R):
    classes = []
    visited = set()
    for i in range(5):
        if A[i] not in visited:
            eq_class = {A[j] for j in range(5) if R[i][j] == 1}
            classes.append(eq_class)
            visited.update(eq_class)
    return classes
```

입력된 관계가 반사,대칭,추이 관계를 모두 만족할 경우 동치 관계로 판정

- "✅ 이 관계는 동치 관계입니다!" 메시지
- 각 원소의 동치류를 화면에 순서대로 출력한다.

구성 조건 (4) 폐포 구현 기능




```python
def closure_transform():
    R = get_matrix()
    if R is None:
        return
    clear_results()

    outer_canvas = tk.Canvas(result_frame, bg="#eef6f9", highlightthickness=0)
    outer_canvas.pack(fill="both", expand=True)

    scrollbar = ttk.Scrollbar(result_frame, orient="vertical", command=outer_canvas.
    yview)
    scrollbar.pack(side="right", fill="y")

    outer_wrapper = tk.Frame(outer_canvas, bg="#eef6f9")
    outer_canvas.create_window((0, 0), window=outer_wrapper, anchor="n")
    inner_frame = tk.Frame(outer_wrapper, bg="#eef6f9")
    inner_frame.pack(anchor="center")

    def on_configure(event):
        outer_canvas.configure(scrollregion=outer_canvas.bbox("all"))
        outer_canvas.itemconfig("all", width=outer_canvas.winfo_width())

    outer_wrapper.bind("<Configure>", on_configure)
    outer_canvas.configure(yscrollcommand=scrollbar.set)
display_matrix_in_center(inner_frame, R, "입력 관계행렬 (Before)")

if not is_reflexive(R):
    R = reflexive_closure(R)
    display_matrix_in_center(inner_frame, R, "반사 폐포 적용 (Reflexive Closure)")
if not is_symmetric(R):
    R = symmetric_closure(R)
    display_matrix_in_center(inner_frame, R, "대칭 폐포 적용 (Symmetric Closure)")
if not is_transitive(R):
    R = transitive_closure(R)
    display_matrix_in_center(inner_frame, R, "추이 폐포 적용 (Transitive Closure)")

tk.Label(inner_frame, text="변환 후 관계행렬 (After)",
         font=("Apple SD Gothic Neo", 10, "bold"),
         bg="#eef6f9").pack(pady=5)

display_matrix_in_center(inner_frame, R, "최종 폐포")

if is_reflexive(R) and is_symmetric(R) and is_transitive(R):
    tk.Label(inner_frame, text="✅ 폐포 적용 후 동치 관계입니다.",
             fg="green", font=("Apple SD Gothic Neo", 10, "bold"),
             bg="#eef6f9").pack(pady=5)
    classes = equivalence_classes(R)
    for i, eq in enumerate(classes):
        tk.Label(inner_frame, text=f"{A[i]}의 동치류: {eq}",
                 font=("Apple SD Gothic Neo", 11),
                 bg="#eef6f9").pack()
else:
    tk.Label(inner_frame, text="❌ 폐포 적용 후에도 동치 관계가 아닙니다.",
             fg="red", font=("Apple SD Gothic Neo", 10, "bold"),
             bg="#eef6f9").pack(pady=5)
```

입력된 관계의 만족 여부에 따라 변환 수행

구성 조건 (4) 폐포 구현 기능

입력된 관계의 만족 여부에 따라 변환 수행

1. 만족할 경우

- 추가 변환 없이 ✅ 이미 동치 관계입니다 메시지 출력
- 현재 행렬을 그대로 출력
- 각 원소의 동치류 출력

구성 조건 (4) 폐포 구현 기능

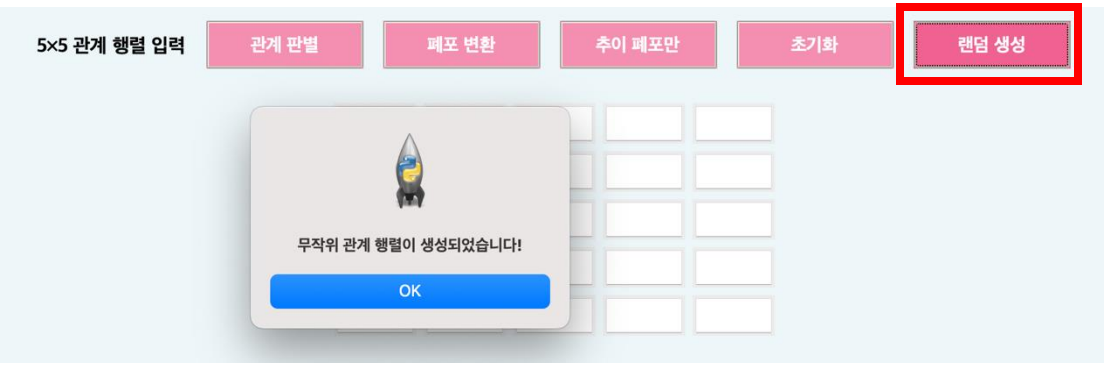
입력된 관계의 만족 여부에 따라 변환 수행

2. 만족하지 않을 경우

각각의 폐포(반사·대칭·추이)를 단계적으로 적용하여

- 변환 전·후 행렬을 출력
- 최종적으로 변환된 관계가 동치 관계인지 다시 판별하여 결과 표시
- 동치류 표시

추가 기능 (1) 행렬 랜덤 생성 기능



```python
def random_relation():
    for i in range(5):
        for j in range(5):
            matrix_entries[i][j].delete(0, tk.END)
            matrix_entries[i][j].insert(0, str(random.randint(0, 1)))
    clear_results()
    messagebox.showinfo("랜덤 행렬 생성 완료", "무작위 관계 행렬이 생성되었습니다!")
```

사용자가 직접 입력하지 않아도 5×5 무작위 관계 행렬을 자동으로 생성해 입력창에 표시하는 기능

: 다양한 관계 예시를 빠르게 테스트하고 프로그램의 판별 및 폐포 기능을 손쉽게 확인할 수 있도록 한다.

추가 기능 (2) 추이 폐포 판단 기능

```python
def show_transitive_closure():
    R = get_matrix()
    if R is None:
        return
    clear_results()
    display_matrix(R, "입력 관계행렬 (R)")
    if is_transitive(R):
        tk.Label(result_frame, text="이미 추이적 관계입니다.",
                 font=("Apple SD Gothic Neo", 11, "bold"),
                 bg="#eef6f9", fg="green").pack(pady=5)
    else:
        R_plus = transitive_closure(R)
        display_matrix(R_plus, "추이 폐포 (R⁺)")
        tk.Label(result_frame, text="✅ 추이 폐포 R⁺가 계산되었습니다.",
                 font=("Apple SD Gothic Neo", 11, "bold"),
                 bg="#eef6f9", fg="blue").pack(pady=5)
```

- **입력된 관계가 추이적인지 여부를 따로 판별**

: 추이 관계가 아닐 경우에는 추이 폐포를 계산하고 그 결과 행렬을 화면에 출력

→ 사용자가 관계의 추이성만 별도로 확인하거나 시각적 비교를 돕는 보조 기능

테스트(1)

1	0	0	1	0
0	1	0	1	0
1	0	0	1	0
0	0	0	0	1
0	0	0	0	0

반사적	❌ False	(3,3), (4,4), (5,5)가 0이므로 반사적 아님
대칭적	❌ False	(1,4)=1인데 (4,1)=0 → 대칭 위배
추이적	❌ False	(1→4)=1, (4→5)=1인데 (1→5)=0 → 추이 위배

반사성 위배: 일부 원소가 자기 자신과 연결되지 않음.

대칭성 위배: (1,4)는 연결되지만 (4,1)은 연결되지 않음.

추이성 위배: (1→4), (4→5)는 존재하지만 (1→5)가 빠져 있음.

반사, 대칭, 추이 중 하나라도 만족하지 않으면 동치 관계가 될 수 없음.

1	0	0	1	0
0	1	0	1	0
1	0	0	1	0
0	0	0	0	1
0	0	0	0	0

반사적: False
대칭적: False
추이적: False

❌ 동치 관계가 아닙니다.

입력 관계행렬 (Before)

1	0	0	1	0
0	1	0	1	0
1	0	0	1	0
0	0	0	0	1
0	0	0	0	0

반사 폐포 적용 (Reflexive Closure)

1	0	0	1	0
0	1	0	1	0
1	0	1	1	0
0	0	0	1	1
0	0	0	0	1

대칭 폐포 적용 (Symmetric Closure)

1	0	1	1	0
0	1	0	1	0
1	0	1	1	0
1	1	1	1	1
0	0	0	1	1

추이 폐포 적용 (Transitive Closure)

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

변환 후 관계행렬 (After)

최종 폐포

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

✅ 폐포 적용 후 동치 관계입니다.

1의 동치류: {1, 2, 3, 4, 5}

입력 관계행렬 (R)

1	0	0	1	0
0	1	0	1	0
1	0	0	1	0
0	0	0	0	1
0	0	0	0	0

추이 폐포 (R^+)

1	0	0	1	1
0	1	0	1	1
1	0	0	1	1
0	0	0	0	1
0	0	0	0	0

✅ 추이 폐포 R^+가 계산되었습니다.

테스트(2)

1	1	1	1	0
0	0	1	1	1
1	1	1	1	0
0	0	0	0	0
1	0	0	0	0

반사적	❌ False	대각선에 0이 있음 (예: (2,2), (4,4), (5,5))
대칭적	❌ False	예: (1,2)=1인데 (2,1)=0 이므로 대칭 아님
추이적	❌ False	예: (1→2) & (2→5)인데 (1→5)=0 → 추이 위배

반사, 대칭, 추이 중 하나라도 만족하지 않으면 동치 관계가 될 수 없음.

1	1	1	1	0
0	0	1	1	1
1	1	1	1	0
0	0	0	0	0
1	0	0	0	0

반사적: False
대칭적: False
추이적: False

❌ 동치 관계가 아닙니다.

입력 관계행렬 (Before)

1	1	1	1	0
0	0	1	1	1
1	1	1	1	0
0	0	0	0	0
1	0	0	0	0

반사 폐포 적용 (Reflexive Closure)

1	1	1	1	0
0	1	1	1	1
1	1	1	1	0
0	0	0	1	0
1	0	0	0	1

대칭 폐포 적용 (Symmetric Closure)

1	1	1	1	1
1	1	1	1	1
1	1	1	1	0
1	1	1	1	0
1	1	0	0	1

추이 폐포 적용 (Transitive Closure)

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

변환 후 관계행렬 (After)

최종 폐포

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

✅ 폐포 적용 후 동치 관계입니다.

1의 동치류: {1, 2, 3, 4, 5}

입력 관계행렬 (R)

1	1	1	1	0
0	0	1	1	1
1	1	1	1	0
0	0	0	0	0
1	0	0	0	0

추이 폐포 (R^+)

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
0	0	0	0	0
1	1	1	1	1

✅ 추이 폐포 R^+가 계산되었습니다.

테스트(3)

1	0	1	0	1
0	1	0	1	0
1	0	1	0	1
0	1	0	1	0
1	0	1	0	1

반사적	✅	모든 대각선 요소가 1임 (R[i][i] = 1)
대칭적	✅	R[i][j] = R[j][i] (예: (1,3)=1, (3,1)=1)
추이적	✅	1~~3~~5는 연결되어 있고, 각 경로가 이미 포함됨

- [1] = {1, 3, 5}
- [2] = {2, 4}
- [3] = {1, 3, 5}
- [4] = {2, 4}
- [5] = {1, 3, 5}

1	0	1	0	1
0	1	0	1	0
1	0	1	0	1
0	1	0	1	0
1	0	1	0	1

반사적: True
대칭적: True
추이적: True

✅ **이 관계는 동치 관계입니다!**

1의 동치류: {1, 3, 5}

2의 동치류: {2, 4}

입력 관계행렬 (Before)

1	0	1	0	1
0	1	0	1	0
1	0	1	0	1
0	1	0	1	0
1	0	1	0	1

변환 후 관계행렬 (After)

최종 폐포

1	0	1	0	1
0	1	0	1	0
1	0	1	0	1
0	1	0	1	0
1	0	1	0	1

✅ **폐포 적용 후 동치 관계입니다.**

1의 동치류: {1, 3, 5}

2의 동치류: {2, 4}

입력 관계행렬 (R)

1	0	1	0	1
0	1	0	1	0
1	0	1	0	1
0	1	0	1	0
1	0	1	0	1

이미 추이적 관계입니다.